



PRACE

Big Data Analysis with Apache Spark



27-28 Nov, 2019

CSC – IT Center for Science Ltd., Espoo

Apurva Nandan
Anni Pyysing



All material (C) 2011–2019 by CSC – IT Center for Science Ltd.
This work is licensed under a **Creative Commons Attribution-ShareAlike**
4.0 Unported License, <http://creativecommons.org/licenses/by-sa/4.0>

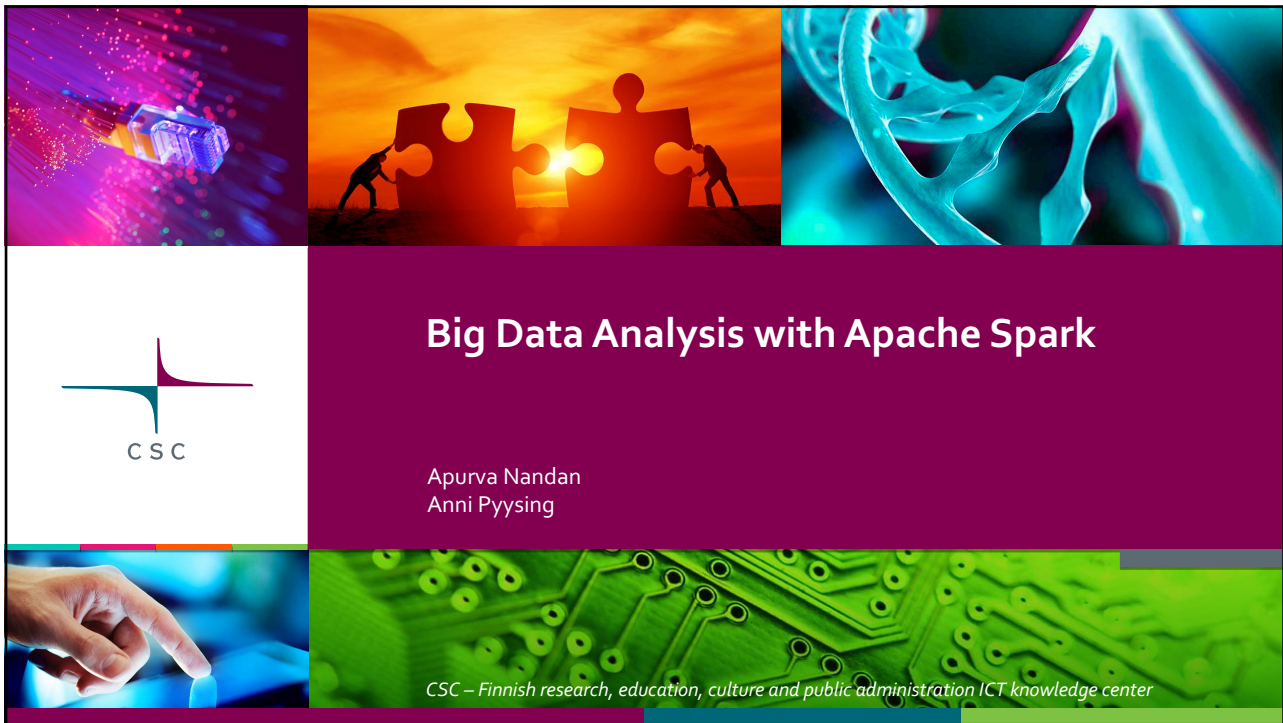


Course schedule

	Wed		Thu
9.00-9.45	Overview and architecture of Spark	9.00-9.30	Spark Dataframes and SQL overview
9.45-10.30	Basics of RDDs + Demo	9.30-10.15	Exercises
10.30-10.45	Coffee break	10.15-10.30	Coffee break
10.45-11.30	RDD: Transformations and Actions	10.30-10.45	Dataframes and SQL contd.
11.00-12.00	Exercises	10.45-12.00	Exercises
12.00-13.00	Lunch break	12.00-13.00	Lunch break
13.00-13.30	Word Count Example	13.00-13.30	Exercises contd.
13.30-14.00	Exercises	13.30-14.00	Brief overview of Spark Streaming and Exercises
14:00-14.15	Coffee Break	14.00-14.15	Coffee Break
14.15-14.30	Short overview of Spark's Machine learning library	14.15-14.30	Demo: Processing Twitter stream with Spark Streaming
14.30-15.30	Exercises	14.30-14.45	Summary of the course
15.30-15.45	Further topics	14.45-15.00	Setting up a Spark Cluster
15.45-16.00	Summary of first day	15.00-16.00	Spark Cluster - UI and Driver

Wireless accounts for CSC-guest network behind the badges. Alternatively, use the eduroam network with your university accounts or the LAN cables on the tables. Accounts to Sisu supercomputer delivered separately.





 CSC

Big Data Analysis with Apache Spark

Apurva Nandan
Anni Pyysing

CSC – Finnish research, education, culture and public administration ICT knowledge center

Before we begin



- Spark is relatively new and rapidly growing over time
- Community Support is getting better and better
- It is very vast and very different to traditional programming approach
- It may take a while and a lot of practice to get used to the Spark 'world'
- In this course, we focus on the basic concepts of Spark from a users point of view.
 - Writing spark code is the primary aspect here and setting up the infrastructure is secondary
- Python is used throughout the course, at least basic knowledge is required
 - Most of the things are Spark related, which run in a pythonic syntax

Agenda



- Day1:
 - ✓ Spark Basics
 - ✓ Machine Learning in Spark
- Day2:
 - ✓ Spark Dataframes
 - ✓ Using Spark with SQL
 - ✓ Spark Streaming
 - ✓ Playing around with Clusters

Basics of the basics



- Processor :
 - ✓ How quickly can you computer process the data
 - ✓ Made up of multiple '**CORES**'
 - ✓ Intel Core i5 processor : 4 cores, Intel Core i7 :4-8 cores
- RAM (**MEMORY**):
 - ✓ Temporary storage of the data in the computer
 - ✓ Allows the processor to make use of the data quickly
 - ✓ Much faster than reading the data from the hard drive
 - ✓ *If you can manage to store the data in the memory, the computation will be much faster*
- NOT ENOUGH RAM IN YOUR LAPTOP :(



1 GB text file
100 GB text file
10 TB text file



Analyze this



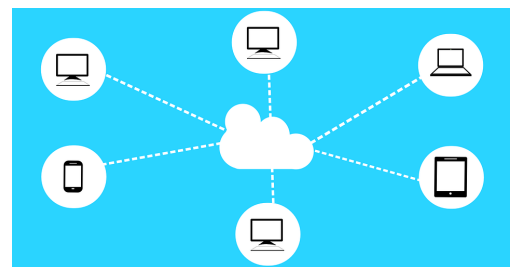
So...Spa...r...k..
Wait!

Brief introduction to Cloud Computing

What is cloud computing



- Computing resources accessed from virtual online 'cloud' rather than your own laptop or PC
- **Virtual Machine(VM)** – machine running in cloud, can be accessed by laptop/PC
- Why should you care?
 - Because everyone is talking about it
- it's a *buzzword*! Yay!
 - It makes your computing life easier



Benefits



- With more resources comes great computing power
 - Think beyond a high performing laptop or PC!
- Reduced Costs – Pay per usage – No need to worry about hardware
- Work from anywhere – Just need an internet connection
- Collaborate better with others – Multiple users on one machine
- Security

Cloud providers

- CSC - cPouta (pouta.csc.fi)
- Amazon - AWS
- Google - Google Cloud Platform
- Microsoft - Azure



Spark - Overview



Apache Spark



- Apache Spark is a fast, Open Source, big data based engine for large scale data analysis and distributed processing
- Developed in Scala and runs on Java Virtual Machine
- Can be Used with Scala, Java, Python or R
- Works on the Map-Reduce concept to do the *distributed processing*
- *Why should you care?*
 - Because DATA • IS GROWING!

Apache Spark: Key Concepts



- **Stores the data into memory when needed, for rapid processing**
- Follows distributed processing concepts for distributing the data across the workers/nodes of the cluster

Spark API

- Scala
- Java
- R
- Python

Cluster Modes

- Standalone
- Yarn
- Mesos
- Kubernetes

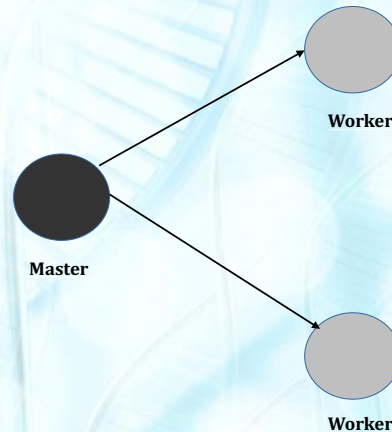
Storage

- File System
- HDFS
- Object Storage
- Kubernetes

Apache Spark: Key Concepts



- **Cluster** : Set of machines grouped together
- **Node**: Individual machine in a cluster
- **Master**:
 - Entry point of a cluster
 - Delegates requests to workers
- **Worker/Slave**: Carries out the processing

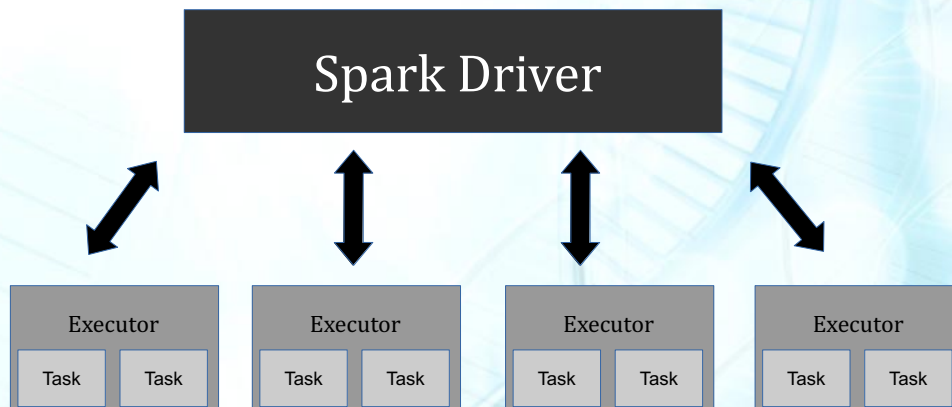


Apache Spark: Key Concepts



- **Spark Job**: Parallel computation consisting of multiple *tasks*
- **Driver** Program: Used to submit your code as a Spark Job to the cluster
- **Executor**:
 - Processes running in Workers
 - Launches individual *tasks* of a Spark Job
 - Each executor uses certain 'Cores' and 'Memory'
- **Task**: Does the actual computation by computing an RDD partition
- **Stage**: Set of parallel tasks – one task per partition. Multiple stages = a spark job
- **RDD**: ?
- **RDD Partition**: ?

Apache Spark: Execution of code



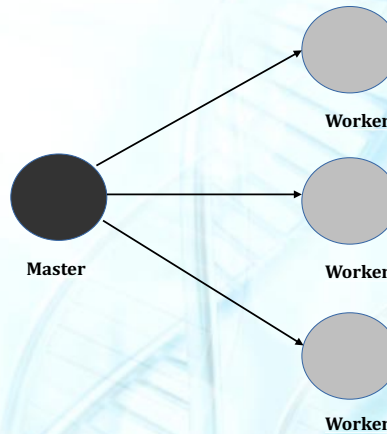
For now, let's assume : each worker is running exactly one executor

Why Run Spark on Cloud?

- You can request as much resource as you want
- You can pay per usage so adding more resources or deleting it would be flexible
- You can access it from anywhere
- You can use the same cluster for a group

Spark on Cloud: Scenario 1

- Worked well with 6GB
- Data increased to 9GB
- Processing time too high
- Add new worker



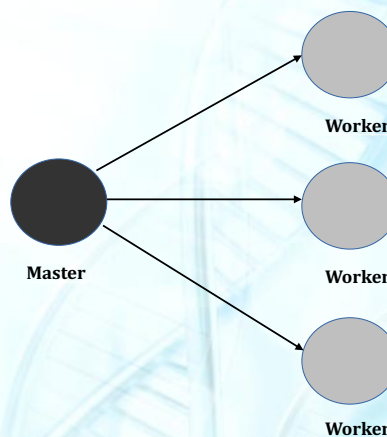
Each Worker has same configuration!

Worker config needs to be decided when deploying the cluster for the first time!

For eg.
Memory: 4GB
Cores: 4

Spark on Cloud: Scenario 2

- Works well with 9GB
- Data decreased to 6GB
- Processing time OK
- Delete Worker, Why?
- Save resources & Cost!



Each Worker has same configuration!

Worker config needs to be decided when deploying the cluster for the first time!

For eg.
Memory: 4GB
Cores: 4

So, What's the point?



- Imagine you have a large VM for the processing and then things happen:
 - You realize you do not all of it, after sometime
 - OR, You need even a bigger VM
 - OR, It fails all of a sudden, ruining everything
- *“Ok, so I will just start with one Worker for now and wait to add more” –*
 - × *Don't!*
 - × *Because....*
 - × *WTF! – Workers Too Fail*

So, Workers or Executors?



- Why use executors at all, when you have workers?
- You might not want to use the whole worker : Eg. Using 2.5 workers out of 3
- Why not remove the worker or get a smaller worker?
- Shared cluster : Others might need some resources!
 - They can just used some part of the worker in form of executors
- **Remember:** *You can always configure the cores and memory of an executor accordingly, but a worker has fixed set of configuration!*

Spark vs Hadoop



- Faster than Hadoop in many cases specially iterative algorithms
- Spark Stores the data in memory which allows faster chaining of operations vs Hadoop Map Reduce which stores the output to disk for any map or reduce operation
- Much more simpler programming API as compared to Hadoop – Less time writing complex map reduce scripts
- Equally good fault tolerance

Who uses Spark?



- Finance
- E-commerce
- Healthcare
- Media & Entertainment
- Travel industry

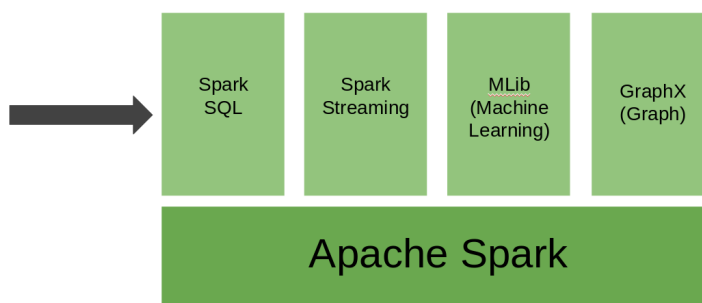
Spark Use Case



- Travel industry
 - Tripadvisor: provides customer recommendations using Spark
 - Every place has 'tags' attached to them
 - Simple yes and no questions are asked first.
 - Recieved from millions of users(*Reliable way of getting information*)
 - For eg. does this hotel have a pool?

Source: <http://engineering.tripadvisor.com/using-apache-spark-for-massively-parallel-nlp/>

Spark Stack





Spark Programming



Python: Basic Data Structures

A = [1, 2, 3, 4, 5]



[[1, 2], [3, 4, 5]]

B = (1, 2, 3, 4, 5)



C = {'key1': 'value1', 'key2': 'value2'}



A[0] = 1

B[4] = 5

C['key1'] = value1

SparkContext



- Main entry point of a spark application
- SparkContext sets up everything and establishes a connection for writing Spark code
- Used to create RDDs, access Spark services and run Spark jobs.

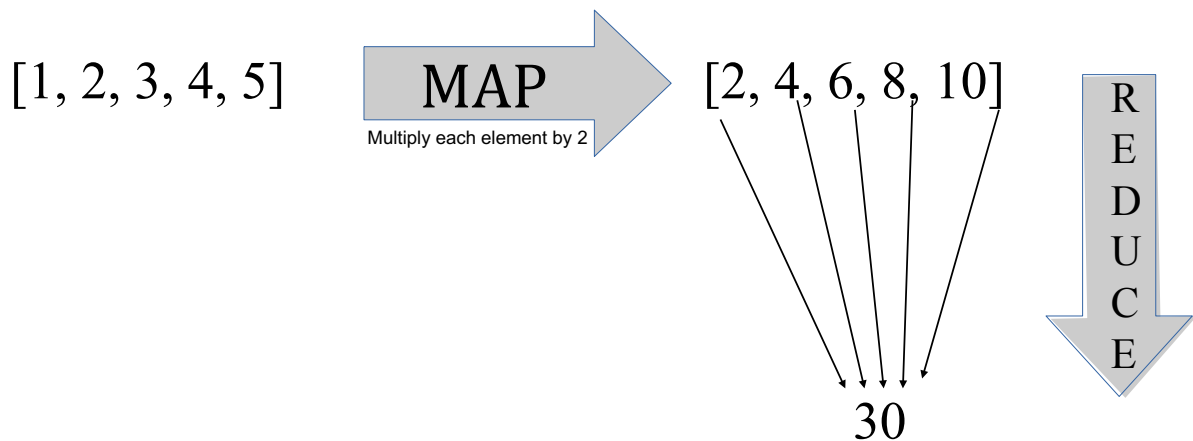
```
>>> from pyspark import SparkContext  
>>> sc = SparkContext()
```

Map-Reduce Paradigm



- Programming model developed by Google
- As the name suggests, there are two phases for processing the data – Map and Reduce
 - Map: Transform each element in the data set
 - Reduce: Combine the data according to some criteria
- Spark uses the Map Reduce programming model for distributed processing

Map-Reduce Paradigm



Spark: RDD



- RDD a.k.a Resilient Distributed Datasets
- Spark's fault tolerant dataset which can be operated in a distributed manner
 - Running the processing across all the nodes of a cluster
- Can be used to store any type of element in it
- *Lazy Evaluation* with all the transformations
- Some Special types of RDDs:
 - Double RDD: for storing numerical data
 - Pair RDD: For storing key pair values

Spark: Pair RDD



- Special form of RDDs which are used for map reduce based functions
- They are in a form of key/value pairs $\langle K, V \rangle$.
 - Key and Value can be of any data type, for eg. String, Integer
- They are used for running functions present in the spark API
- Generally, Pair RDDs are used more often

Spark: Creating RDDs



- By distributing the native python collection:

```
>>> sc.parallelize(['this', 'is', 'an', 'example'])  
>>> sc.parallelize(xrange(1,100))
```
- By reading text files from local file system , HDFS or object storage:

```
>>> sc.textFile('file:///example.txt')  
>>> sc.textFile('hdfs:///hdfsexample.txt')
```
- This output RDD is an RDD of strings

Spark: RDD Transformations



- **map(func)** :Return a new RDD formed by passing each element of the source through a function func.
- **filter(func)** :Return a new RDD formed by selecting elements of the source on which func returns true.
- **flatMap(func)** : Similar to map, but flattens the final result.
- **groupByKey([numTasks])** :When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs.
Note: If you are grouping in order to perform an aggregation (such as a sum or average) over each key, using reduceByKey or aggregateByKey will yield much better performance.
- **reduceByKey(func, [numTasks])** :When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function func, which must be of type (V,V) => V.
- **sortByKey([ascending], [numTasks])** : When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean ascending argument.
- **join(otherDataset, [numTasks])** : When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (V, W)) pairs with all pairs of elements for each key. Outer joins are also supported
- **cogroup(otherDataset, [numTasks])** : When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (Iterable<V>, Iterable<W>)) tuples. This operation is also called groupWith.

All Spark Transformations are Lazy!

Spark: RDD Actions



- **collect()** : Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
- **count()** :Return the number of elements in the dataset.
- **first()** :Return the first element of the dataset (similar to take(1)).
- **take(n)** :Return an array with the first n elements of the dataset.
- **takeSample(withReplacement, num, [seed])** : Return an array with a random sample of num elements of the dataset, with or without replacement, optionally pre-specifying a random number generator seed.
- **takeOrdered(n, [ordering])** : Return the first n elements of the RDD using either their natural order or a custom comparator.
- **saveAsTextFile(path)** : Write the elements of the dataset as a text file (or set of text files) in the local filesystem, HDFS
- **CountByKey()** : Only available on RDDs of type (K, V). Returns a hashmap of (K, Int) pairs with the count of each key.
- **foreach(func)** : Run a function func on each element of the dataset. This is usually done for side effects

Actions are Eager!

Spark: Lambda Expressions



- Lambda expressions: simple functions that can be written as an expression.
- Do not support multi-statement functions or statements that do not return a value
- Very useful in Spark transformations. *Why?*
- For eg. **lambda x: x*x**
- *Will take x as an input and return x^2 as the output*

```
def function(x):
    return x*x
```

RDD: Transformations - Map



- Used to apply a function to each element in the RDD
- This is used to modify elements of the RDD. One could completely change the format of the RDD
- `>>> rdd = sc.parallelize(range(1,10)) # [1,2,.....10]`
- `>>> rdd_map = rdd.map(lambda x: (x, x))`
- `>>> rdd_map.collect() # [(1,1), (2,2) ..... (10,10)]`
- `>>> rdd_map = rdd_map.map(lambda x: (x[0]*2, x[1])).collect()`
- Special case: flatMap:
- `>>> rdd_flatmap = rdd_map.flatMap(lambda x: (x[0], x[1] + 1))`

RDD: Transformations - Filter



- Used to filter out the RDD according to a defined criteria

- ```
>>> rdd = sc.textFile('HP.txt')
```

the place where things are hidden

if you have to ask you will never know

if you know you need only ask

- ```
>>> rdd = rdd.filter(lambda x: x.startswith('the'))
```

the place where things are hidden

RDD: Tabular to PairRDD



Always remember, if your RDD turns out to be something like this:

col1, col2, col3

1,[1,2],'hello'

2,[3,4],'moi'

3,[5,6],'hola'

4,[7,8],'ciao'

Convert it into a key/pair format like:

1, ([1,2], 'hello')

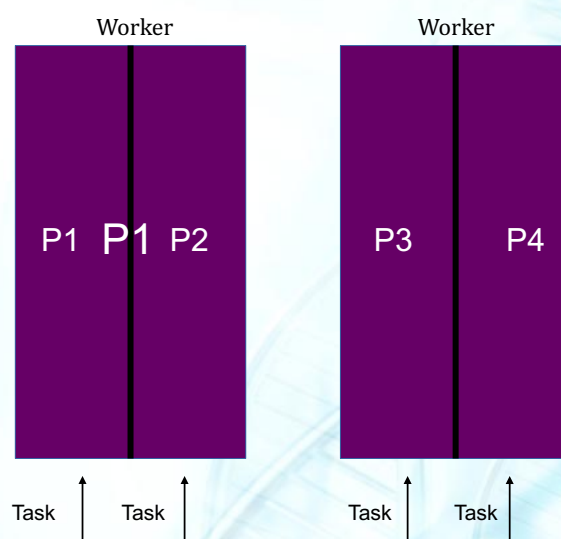
-
- This means, now you have one key and the rest of the things are values. This helps Spark operate the RDD as a PairRDD. (Also, reduceByKey, countByKey will get their 'Key' ;-)

Spark: Parallelism - Partitions



- Spark allows the user to control parallelism by providing partitions when creating an RDD or changing the configuration
- *Example: >>>sc.textFile(inputfile, numPartitions)*
- **Partitioning** : accessing all hardware resources while executing a Job.
- More Partition = More Parallelism
- For each partition – One task is launched

Spark: Parallelism - Partitions



Spark: Parallelism



- Spark can only run 1 concurrent task for every partition of an RDD, up to the number of cores in your cluster.
- If you have a cluster with 50 cores, you want your RDDs to have 50 partitions and then can try to increase it 2 to 3 times more.
- For choosing "good" number of partitions : At least as many as the number of executors for parallelism.
- *User should check the hardware for how many tasks can run on each executor. Get a balance between under utilizing the cluster and over utilizing it*

Example: Word Count



- Count the occurrences of words across all lines

if you have to ask you will never know
if you know you need only ask



you 4
if 2
ask 2
to 2
know 2
have 1
will 1
never 1
need 1
only 1

Example: Word Count



```
# Load the text file
>>> rdd = sc.textFile('HP.txt')
# Split the text into words and flatten the results. Why?
>>> words = rdd.flatMap(lambda line: line.split())
>>> words.collect()
('if' , 'you', 'have', 'to', 'ask', 'you' 'will', 'never', 'know', 'if', 'you', 'know' , 'you' ,
'need' , 'only', 'ask')
```

Example: Word Count



```
# Load the text file
>>> rdd = sc.textFile('HP.txt')
# Split the text into words and flatten the results. Why?
>>> words = rdd.flatMap(lambda line: line.split())

>>> words_map = words.map(lambda x: (x,1))
(('if',1) , ('you',1), ('have',1), ('to',1), ('ask',1), ('you',1), ('will',1), ('never',1),
('know',1), ('if',1), ('you',1), ('know',1) , ('you',1) , ('need',1) , ('only',1), ('to',1),
('ask',1))
```



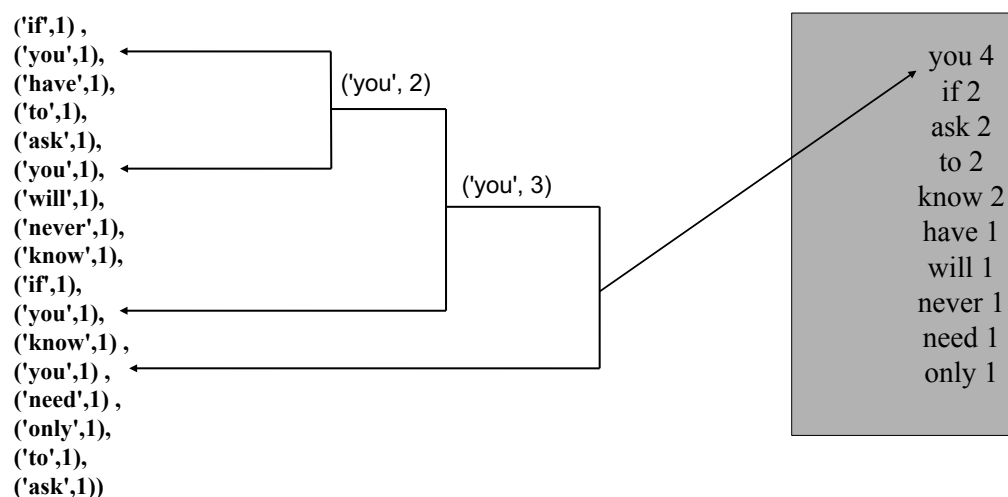
Example: Word Count

```
# Load the text file
>>> rdd = sc.textFile('HP.txt')
# Split the text into words and flatten the results. Why?
>>> words = rdd.flatMap(lambda line: line.split())

>>> words_map = words.map(lambda x: (x,1))
(('if',1), ('you',1), ('have',1), ('to',1), ('ask',1), ('you',1), ('will',1), ('never',1),
('know',1), ('if',1), ('you',1), ('know',1), ('you',1), ('need',1), ('only',1), ('to',1),
('ask',1))

>>> words_count = words_map.reduceByKey(lambda a,b: a+b)
```

Example: Word Count



RDD: Shuffles



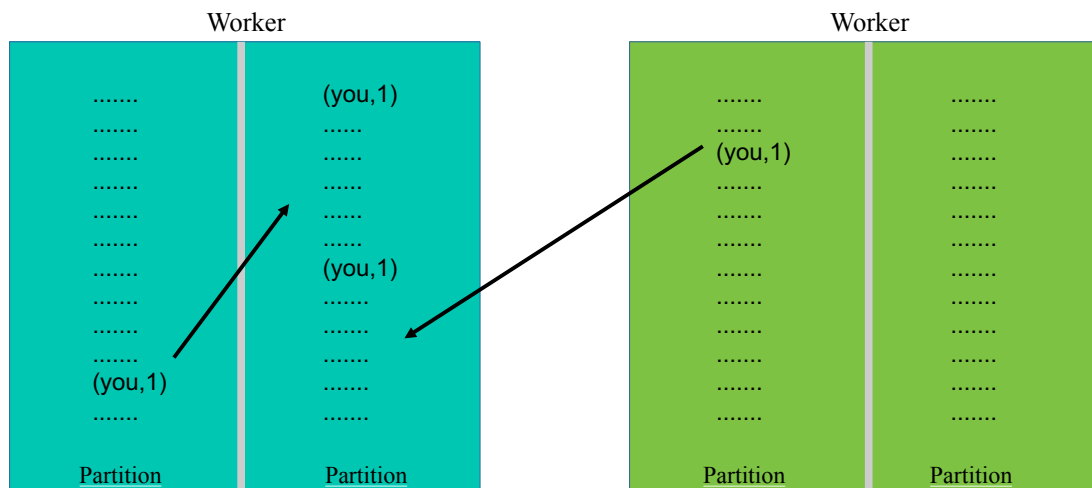
- Happens for some Spark transformations like *groupByKey* and *reduceByKey*.
- Data required in computations of records can be in different partitions of the parent RDD.
- The tasks require that all the **tuples with same key have to be in the same partition** and should be processed by the same task.
- Spark executes a **shuffle, which involves transfer of data across the workers** of the cluster
 - This results in a new stage with completely new set of partitions.

RDD: Shuffles



- Expensive operations as they *incur heavy disk and network I/O*
- Primary goal when choosing an arrangement of operators is to *reduce the number of shuffles and the amount of data shuffled*.
 - Because shuffles are expensive.
- All the shuffle data is written to disk and transferred over the network.
- *join, cogroup, *By or *ByKey* transformations result in shuffles.
 - Each of these operations have different costs
 - As a novice one has to be careful when selecting one over another

RDD: Shuffles



Repartitioning



- Used for changing the number of partitions in RDD or DF
- Why change the partitions?
- **Repartition :**
 - Tries to shuffle the data all over the network to increase/decrease the number of partitions.
 - Any shuffle operation is costly when it comes to large operations.
- **Coalesce:**
 - Tries to combine the number of partitions without performing the costly shuffle.
 - Can be only used to decrease the number of partitions
- **partitionBy:**
 - Custom partitions
 - Makes shuffling functions more efficient i.e reduceByKey(), join(), cogroup() etc

Cache / Persist



- For storing the RDD in order to do faster operations (approx 10x)
- Avoids recomputation of partitions already computed
- For persist there are many options like (not a comprehensive list):
 - MEMORY_ONLY
 - MEMORY_AND_DISK
 - DISK_ONLY
- cache() is basically persist() but with default value of: MEMORY_ONLY
- MEMORY_ONLY tries to store everything in memory, if it cannot store it all, some partitions are computed on the fly
- MEMORY_AND_DISK: Try to store in memory, if the RDD doesn't fit, store the rest on disk
- There are advanced serialization storage options as well. (More CPU intensive)

Machine learning in Spark



Spark Mllib – Machine learning in Spark



- Regression
- Classification: Problem of identifying to which set of categories , a new observation belongs, on the basis of a training set of data containing observations (or instances) whose category membership is known.
- Example :
 - assigning a given email into "spam" or "non-spam" classes
 - assigning a diagnosis to a given patient as described by observed characteristics of the patient (gender, blood pressure, presence or absence of certain symptoms, etc.).
- Clustering: The task of grouping a set of objects in such a way that objects in the same group (called a cluster) are more similar (in some sense or another) to each other than to those in other groups (clusters).

03/02/15

53

Spark Mllib – Machine learning in Spark



- Train: Learn from the given data
- Predict : Predict output for an unseen data
- Types of data sets:
 - Training Set: The data set used in training the algorithm
 - Test set: The data set on which we need to predict the output

FEATURES AND OUTPUT LABEL

The data on the right side shows “features” and output label in a dataset, which shows selection of students in a particular university.

Gre gpa and rank are the features which determine if a student will get into the university or not. Admit is the output label which denotes 1 for True (Accepted) and 0 for False

gre	gpa	rank	admit
380	3.61	3	0
660	3.67	3	1
800	4.00	1	1
640	3.19	4	1
520	2.93	4	0
760	3.00	2	1

03/02/15

54

Spark Mllib – Machine learning in Spark



- Apache Spark's scalable machine learning library.
- Used with Spark's APIs and compatible with NumPy in Python
- 100x faster than Hadoop
- Don't reinvent the wheel: If you are looking to do use any of the machine learning based techniques in your work , it is always a best idea to see if Spark has the algorithm in Mllib library
- The algorithms provided are much more optimized and ready to use
- This course does not focus on detailed Machine learning concepts but just gives an overview

Spark Mllib – Machine learning in Spark



- **Classification:** logistic regression, naive Bayes,...
- **Regression:** generalized linear regression, survival regression,...
- **Decision trees,** random forests, and gradient-boosted trees
- **Recommendation:** alternating least squares (ALS)
- **Clustering:** K-means, Gaussian mixtures (GMMs),...
- **Topic modeling:** latent Dirichlet allocation (LDA)
- Frequent itemsets, association rules, and sequential pattern mining
- Feature transformations: standardization, normalization, hashing,...
- ML Pipeline construction
- Model evaluation and hyper-parameter tuning
- ML persistence: saving and loading models and Pipelines
- Other utilities include:
 - Distributed linear algebra: SVD, PCA,...
 - Statistics: summary statistics, hypothesis testing,...



Why Spark SQL?

Calculate AVG : RDD vs DF



```
>>>rdd = sc.parallelize([('Barcelona',2), ('Rome',3),
('Paris',4), ('Vegas',5), ('Barcelona', 8), ('Vegas',9), ('Rome',3)])
```

→ Using Dataframes

Using RDD directly



```
>>>rdd_avg = rdd.map(lambda item: (item[0], [item[1],
1])).reduceByKey(lambda a,b: (a[0] + b[0], a[1] +
b[1])).map(lambda item: (item[0], item[1][0] / item[1][1]))
```

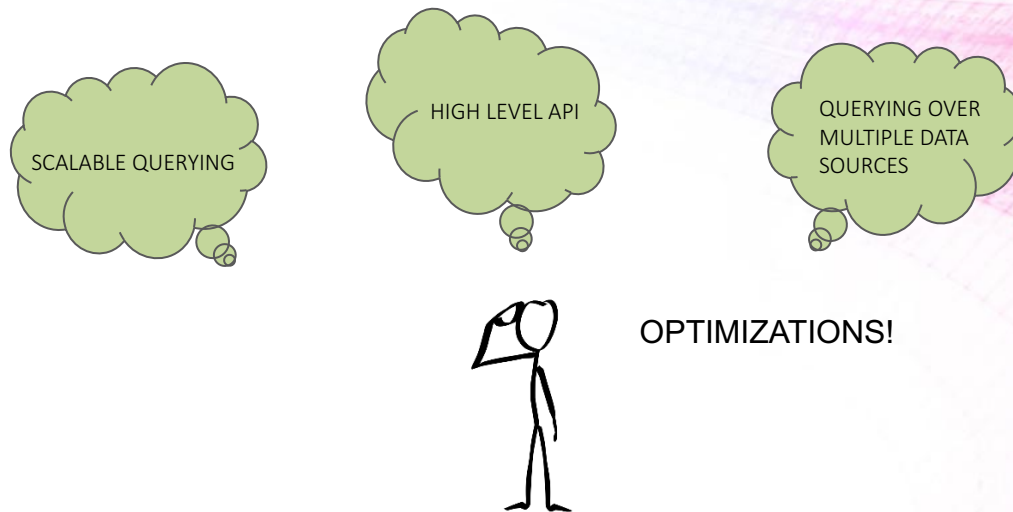
```
>>>rdd_avg.collect()
```

```
>>>rdd_for_df = rdd.map(lambda item:
Row(city=item[0], count=item[1]))
>>>df = rdd_for_df.toDF()
```

```
>>>df_avg = df.groupBy('city').avg('count')
```

```
>>>df_avg.show()
```

Spark SQL



SparkSession

Entry point to Spark SQL.

First object which needs to be created while developing Spark SQL applications

```
>>> from pyspark.sql import SparkSession
>>> spark = SparkSession.builder.getOrCreate()
```

OR

```
>>> spark = SparkSession \
    .builder \
    .appName("Python Spark SQL basic example") \
    .getOrCreate()
```

Spark SQL: Dataframes



- A distributed collection of data organized into named columns.
- Conceptually equivalent to a table in a relational database or a data frame in R/Python, but with richer optimizations under the hood (Catalyst Optimizer)
- Can be constructed from CSV files, JSON files, Parquet files, external databases, Text files, existing RDDs.
- Can be thought of as: RDDs with Schema
- Allows data to be fetched using SQL Statements

Spark SQL: Dataframes



- You can try to fetch the data using SQL queries
 - Example: `doginfo = spark.sql("SELECT name, breed FROM dogview")`
- Or, You can use the functions of the dataframe api.
 - Example: `doginfo = dogdataframe.select("name","breed")`
- Both are optimized by Catalyst optimizer
- Immutable
- In case if you want to perform any complex operation : the dataframe can be transformed back to an RDD, then you could use the core spark functions

Spark SQL: Dataframes



```
>>>df = spark.read.json("people.json")
>>>df.show() # Displays the content
>>>df.printSchema()

# Select only the "name" column
>>>df.select("name").show()

# Write the file as Parquet format
>>>df.write.parquet("people.parquet")
# Read the parquet file
>>>df_people = spark.read.parquet("people.parquet")

# Filter the records based on age
>>>df_people_filtered = df_people.filter(df_people['age'] > 21)
```

Spark SQL: Dataframes



```
# Read the file
>>>lines = sc.textFile("people.txt")
# Split the columns of text file into two
>>>parts = lines.map(lambda l: l.split(","))

# Remove unwanted spaces
>>>people = parts.map(lambda p: (p[0], p[1].strip()))

# Select only the "name" column
>>>df.select("name").show()
```

Spark SQL: Dataframes



```

→ # Infer the schema, and register the DataFrame as a table.
>>schemaPeople = spark.createDataFrame(people)
>>schemaPeople.printSchema()

→ # Declare schema fields with the name of the field and the type of the field
>>>fields = [StructField("name", StringType(), True), StructField("age", IntegerType(), True)]

# Create schema
>>>schema = StructType(fields)

# Applying the generated schema to the RDD.
>>>schemaPeople = sqlContext.createDataFrame(people, schema)
>>>schemaPeople.printSchema()

```

Spark SQL: Dataframes



filter – same as SQL command 'WHERE'
select – same as sql command 'SELECT'
join – joins two based on a given expression
groupBy – Groups the dataframe by specified columns for performing aggregation (min,max,sum,avg,count)
orderBy – sorted by specific cols
union – combining rows of two dataframes
limit – limits result count to specified value

distinct – returns distinct rows in a dataframe
drop – drops a specific col from df
dropDuplicates – duplicate rows removed
fillna – replaces null values
dropna – drops null values
replace – replaces a value in a dataframe with another

rdd – returns the rdd representation of the dataframe
foreach – Applies a function to each record

Spark SQL: Dataframes - Columns



alias – new name for the column

startswith – check if the col value starts with the given string

endswith – check if the col value ends with the given string

contains – contains a string

isNull – check if the col value is null

isNotNull

like – SQL like match

cast – change the column data type

Example:

```
# shows the dogs that have the string "retriever" in the column breed
dogdataframe.filter(dogdataframe.breed.contains("retriever")).show()
```

Spark SQL: Dataframes



- In order to use SQL statements, we create → Temporary view
- Temporary views in Spark SQL are session-scoped
- Disappear if the session that creates it terminates.

Create a temporary view from a dataframe

```
>>> dataframe.createOrReplaceTempView("table_name")
```

now, use the SQL statements

```
>>> result = spark.sql("SELECT * FROM table_name")
```

Spark SQL: Dataframes – SQL Query



- Query statements:
 - SELECT
 - GROUP BY
 - ORDER BY
- Common operators:
 - Relational operators ($=$, $<$, $>$, $>=$, $<=$)
 - Arithmetic operators (+, -, *, /, %)
 - Logical operators (AND, OR, BETWEEN, NOT)
- Joins
 - JOIN
 - {LEFT|RIGHT|FULL} OUTER JOIN
- Unions
- Sub-queries

Spark SQL: Dataframes - Optimization



- Only reads the data which is required by:
- Skipping a partition while reading the data
- Skipping the data using statistics (min, max)
- Using Parquet file format (explained in the next slide)
- Using Partitioning (department=sales)

Apache Parquet

- Compressed , space efficient columnar data format
- Can be used with Spark API for reading and saving dataframes

Read Example: `dataframe = spark.read.parquet('filename.parquet')`

Write Example: `dataframe.write.parquet('filename.parquet')`

- Most preferred way of storing/reading data from/to Spark
- **ONLY reads the columns that are needed rather than reading the whole file. Limiting I/O cost**

Storing large data

- HDFS : Hadoop Distributed File System
 - Java-based file system for scalable and reliable data storage
 - Most widely used storage mechanism
 - Fault-tolerant
- Regular File System (readable by all the workers)
- Object Storage

Object Storage – What?

- Stores "objects"
 - In general you can think of objects as files.
- Data objects stored in buckets (or containers)
 - A bucket basically acts like a filesystem directory
-
- Multiple copies of the data stored, spread across different servers
 - protects the data against disk and server failures.
- Examples: Amazon S3, CSC Pouta Object Storage

Object Storage: Benefits

- No need to set up a virtual machine to serve or receive the data.
- The data can be accessed from anywhere using a URL over the web.
- The data can have different levels of access control
 - Private / Public / Temp URL
- Big data frameworks like Apache Spark can directly read and write data via HTTP
- Less costly than HDFS
- Even better fault tolerance



Spark Streaming

Spark Streaming



Runs **on demand** whenever user requests it to run



Runs **continuously** after a fixed interval of time

Spark Streaming

- Enables scalable, high-throughput, fault-tolerant stream processing of live data streams.
- Data can be ingested from many sources like Kafka, Flume, Kinesis, or TCP sockets
- Can be processed using complex algorithms expressed with high-level functions like map, reduce, join and window.
- Spark Streaming receives live input data streams
- Divides the data into batches
- Processed by the Spark engine to generate the final stream of results in batches.



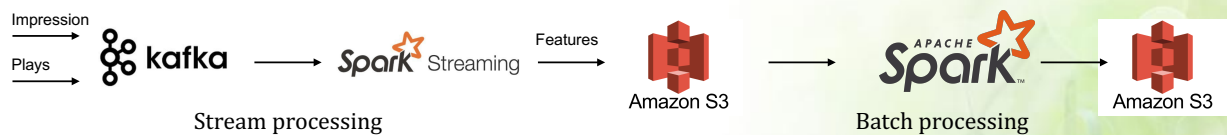
Spark Streaming

- Continuous stream of data is called discretized stream or *Dstream*
- Internally, a DStream is represented as a *sequence of RDDs*.
- Any operation applied on a DStream translates to operations on the underlying RDDs.



Spark Streaming Use Case

- Netflix: provides customer recommendations using Spark
 - New titles are added , Interests of members evolve
 - Relevant titles are recommended to every member
 - Respond to the changing user base, interests etc



Source: <https://www.slideshare.net/databricks/near-realtime-netflix-recommendations-using-apache-spark-streaming-with-nitin-sharma-and-elliott-chow>

Some More Useful Stuff

Setting up your spark cluster



- CSC : Clusters are setup on **rahti.csc.fi** , support email: servicedesk@csc.fi
- Amazon: <https://docs.aws.amazon.com/emr/latest/ReleaseGuide/emr-spark.html>
- Google Cloud: <https://cloud.google.com/dataproc/>
- Microsoft: <https://docs.microsoft.com/en-us/azure/hdinsight/spark/apache-spark-overview>

Apache Spark: Analysis



Submitting Jobs to Spark



```
from pyspark import SparkContext, SparkConf # SparkContext Needs to be called first

conf = SparkConf() # Used to provide optional configurations to the spark application
conf.setAppName('Test app')

sc = SparkContext(conf=conf)
rdd = sc.parallelize(range(1, 10**5))

print(rdd.count())
print(rdd.collect())

# Submit your code : sh $SPARK-HOME/bin/spark-submit filename.py
```

More Parallelism



- The following parameters can be provided as configs to adjust the parallel behavior too
- spark.default.parallelism
 - Works for raw RDDs – parallelize, join, reduceByKey
 - Ignores the dataframes
- spark.sql.shuffle.partitions
 - For dataframes
- Number of partitions that are used when shuffling data for joins or aggregations

Random Thoughts



- Use *reduceByKey* to perform aggregation or counting instead of *groupByKey*
- Avoid printing huge data sets using *rdd.collect()*
- Try to convert the RDD to a dataframe whenever you can, they offer better performance
- Convert the DF back to RDD in case you need complex operations
- Always be sure to check the partitions for parallelization. You might need to scale up or down accordingly
- Try to accomplish the things using Spark's map, flatMap functions when you need to apply something over the elements.



Thank you!

- Spark Cluster Requests: servicedesk@csc.fi
- [Spark Programming Guide](#)
- [PySpark Dataframe Reference \(for functions\)](#)
- Learning Spark: Lightning-Fast Big Data Analysis – Book (O'REILLY)



<https://www.facebook.com/CSCfi>



<https://twitter.com/CSCfi>



<https://www.youtube.com/c/CSCfi>



<https://www.linkedin.com/company/csc---it-center-for-science>

Exercises

General instructions

- Open notebooks.csc.fi
- Login with HAKA (if you have one) or else request for a non-HAKA account
- Go to *Account* tab → *Join Group*
- Enter the joining code show by the instructor
- Go back to the *Dashboard* tab
- Launch the environment called **Apache Spark Course – CSC**
- Wait for the provisioning to complete and then click on “*Open In Browser*”
- Enter the *work* directory -> *spark*
- You should now see the required Jupyter Ipython Notebooks and datasets